Université de Nice-Sophia Antipolis
Master 1 ESTel
Composants logiques programmables

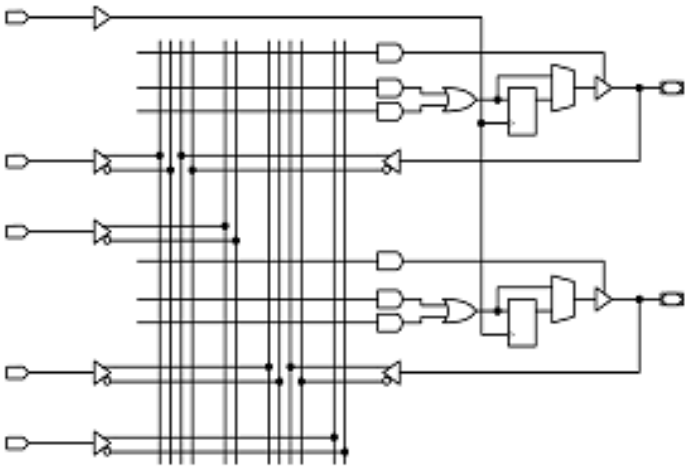
TD1 : Composants logiques programmables simples

Exercice 1 : Technologie des circuits logiques programmables

1 - Expliquer la différence entre une EPROM, un PAL et un PLA.
2 - Indiquer le nombre d'entrée et de sortie du circuit suivant : GAL 16 V 8.
3 - Comparer la surface de silicium occupée pour une technologie bipolaire et MOS.

Exercice 2 : Notion de point de connexion programmable

Redessiner le schéma ci-dessous en faisant apparaitre l'ensemble des fusibles (point de connexion programmable) et la taille véritable des portes logiques. Déterminer le nombre de transistors équivalents en particulier celui de la partie programmable.



Exercice 3 : Opérateurs et Exploration d'Architecture

Additionneurs/soustracteur

4.1 - On propose dans cet exercice d'étudier un additionneur à retenue conditionnelle. Cet additionneur comporte deux parties :
- un additionneur à retenue propagée sur la partie basse,
- deux additionneurs à retenue propagée sur la partie haute comme le montre la figure ci_après.

Université de Nice-Sophia Antipolis
Master 1 ESTel
Composants logiques programmables

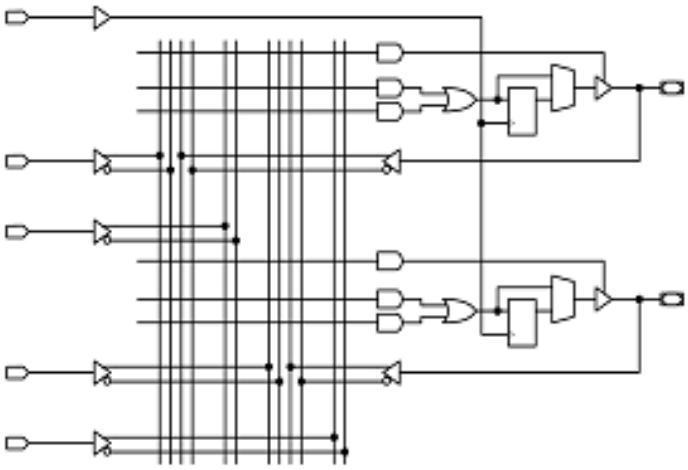
TD1 : Composants logiques programmables simples

Exercice 1 : Technologie des circuits logiques programmables

1 - Expliquer la différence entre une EPROM, un PAL et un PLA.
2 - Indiquer le nombre d'entrée et de sortie du circuit suivant : GAL 16 V 8.
3 - Comparer la surface de silicium occupée pour une technologie bipolaire et MOS.

Exercice 2 : Notion de point de connexion programmable

Redessiner le schéma ci-dessous en faisant apparaitre l'ensemble des fusibles (point de connexion programmable) et la taille véritable des portes logiques. Déterminer le nombre de transistors équivalents en particulier celui de la partie programmable.



Exercice 3 : Opérateurs et Exploration d'Architecture

Additionneurs/soustracteur

4.1 - On propose dans cet exercice d'étudier un additionneur à retenue conditionnelle. Cet additionneur comporte deux parties :
- un additionneur à retenue propagée sur la partie basse,
- deux additionneurs à retenue propagée sur la partie haute comme le montre la figure ci_après.

fiche de synthèse :

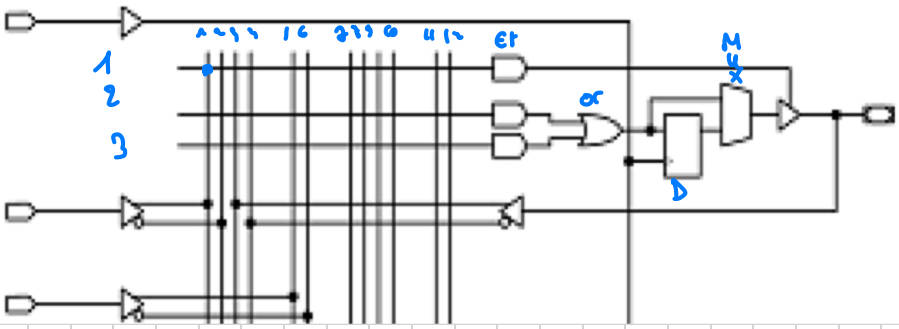
EPROM :  technologie qui mémorise un signal même après coupure.

PLA : Réseau logique programmable :
(plan Et / plan ou) personnalisable

PAL : Réseau logique programmable :
(plan Et / plan ou) non personnalisable que le ou.
le plan Et peut être modifié.

GAL 16v8 : on a comme entrée : 8 bit
Sortie : 8 bit

technologie non comparable entre bipolaire et fets
car l'un plus récent que l'autre et l'autre plus avancé.



Pour compter le nombre de fusible / transistor :
↳ un point bleu (.) contient 3 transistor (SRAM + 2 transistor).
le nombre de point bleu : 3 lignes / 12 colonnes.

$\underbrace{(3 \times 12)}_{\text{SRAM}} \times \underbrace{3}_{\text{logic}}$ trs = 108 trs

on trouve 108 trs pour les éléments programmable.

- pour le Et : un Et est constitué de (4+6×2) trs.
  on en compte 3. donc 3(4+20) ⇒ 72 trs
- pour les ou : un ou est constitué de 2 trs.
  on compte 2 entrées ou : (2×2) 4 trs.
- pour un Nux : 2 trs.
- pour une bascule D : 10 trs
- pour un Buffers : 3 trs

on compte 195 trs pour ce circuit.

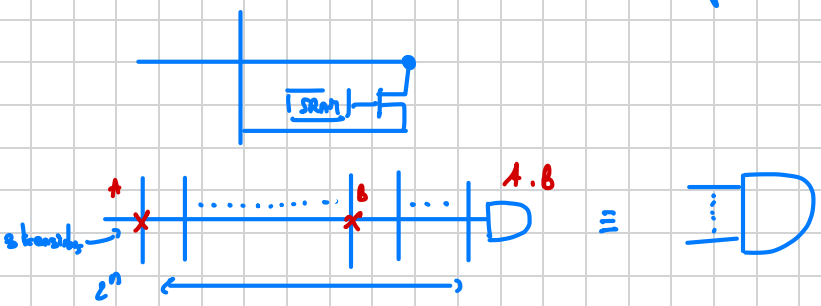
Pour un Add 32 bits : → entrée 64.
sortie 32.

Add : $S_i = A_i \oplus B_i \oplus R_{i-1}$
$= A_i.B_i.R_{i-1} + A_i.B_i.R_{i-1} + A_i.B_i.R_{i-1} + A_i.B_i.R_{i-1}$) 1 cell.
$= R_i = A_iB_i + A_iR_{i-1} + B_iR_{i-1}$) 1 cell

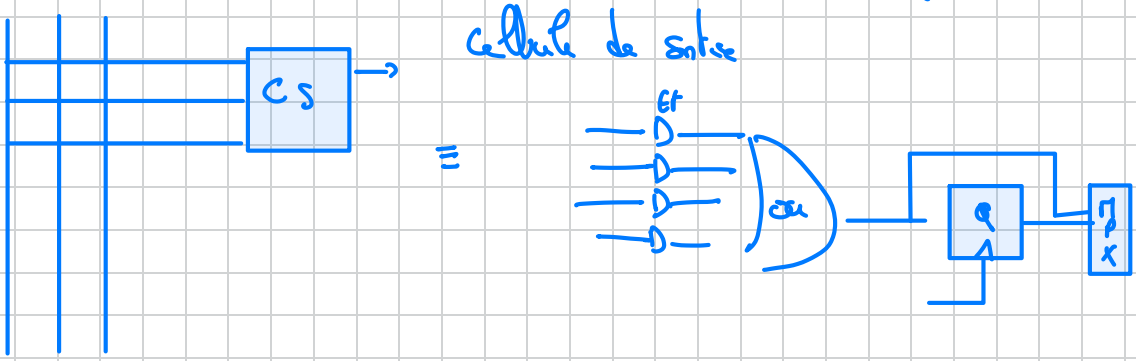
Nb de cellule : si $(P < ou = N)$ alors $NC = 1$
si $P > N$ alors $NC = \frac{(P-N)}{(N-1)} + 1$

Exercice 1 :

1) Ensemble d'éléments mémoire pour récuperer les signaux d'entrée : Et.



2) Matrice programable : (PAL) → Matrice figé.



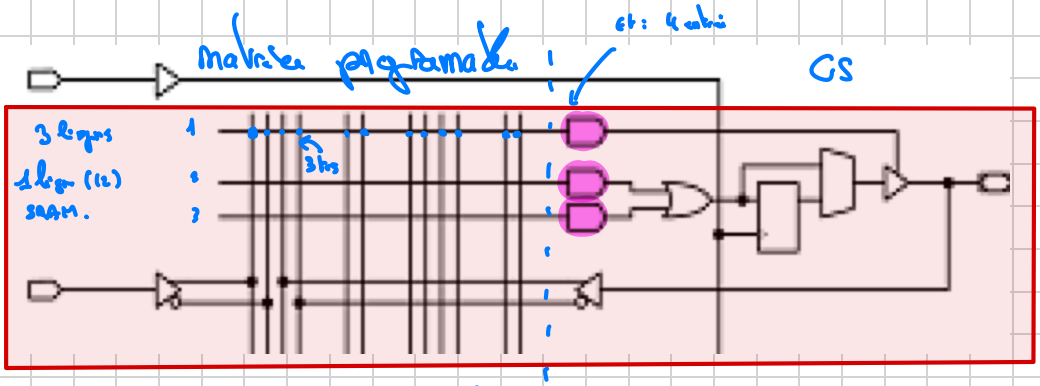
Cellule de sortie

Matrice ou Programable. (PLA)

3). GAL 16V8 : ← 8 s et 8 E.

4). bipolaire avant → CMOS maintenant. | donc pas de comparaison valable (non évoqué ici).

Exercice 2 :



3 trs car 1 SRAM + 2 trs.

- Eléments programable (SRAM) $\frac{(3\times12)}{\text{SRAM}} \times \frac{3 \text{ trs}}{\text{ligne}} = 108$ trs
- Porte And : $\frac{3}{\text{lig}} \times (\underbrace{4+10\times2}_{\text{Et}}) = 72$ trs
- Bascule D : 10 trs
- Mux : 2 trs
- Buffers : 3 trs.

- ou : $(\underbrace{2\times2}_{\text{subou}}) = 4$ trs.

1 SRAM → 2 trs

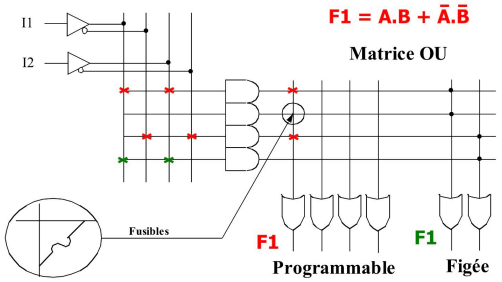
Et/ou → 2 in → 4 trs + 1 ns + 2 ns

Bascule D : 10 trs

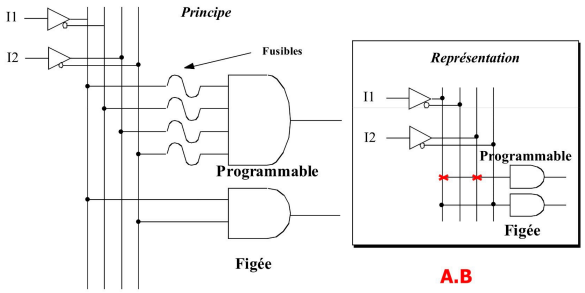
Mux 2 : 1 → 2 trs

Buffers : 1 trs.

La Matrice OU



La Matrice ET

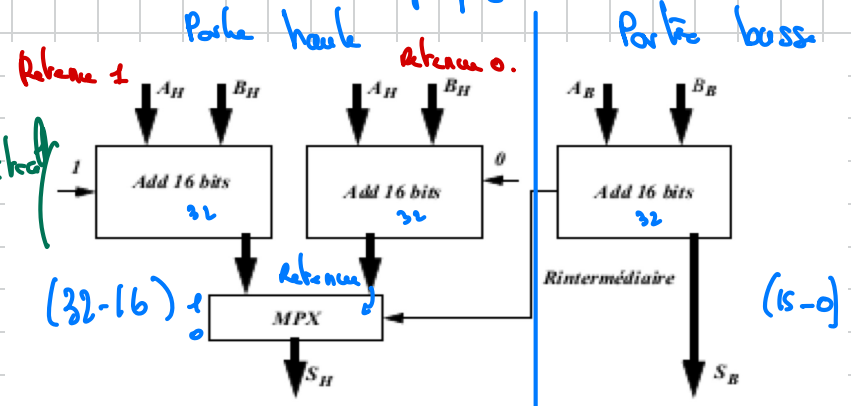


Exercice 3 :

Add retenue propagé. (lent)

Add : 32 bits à retenue propagé : $t_r = 32 \times t_c$ cell. → 64 cells. Surface

A conditionnelle.



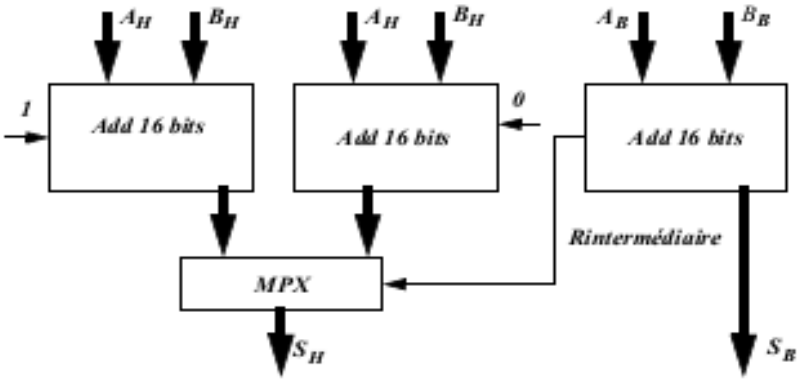
Add 16 bits. → 48 bits. x2 = 96 cell

mpx → 16 x2 = 32 cell
(16:2→1)

} 128 cells. Surface.

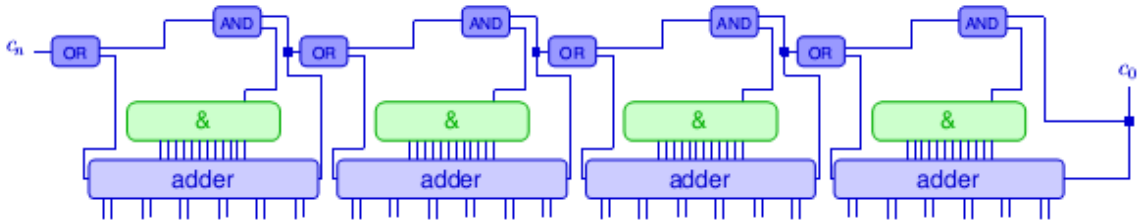
A add avec A conditionnelle.

add → 16 x tc cell
mpx → 1 x tc cell
} 17 x tc cell.



Pour un additionneur 32 bits, donner les équations du système puis calculer le temps de réponse et le nombre de portes logiques. Comparer avec les architectures étudiées en cours.

4.2 – Rependre la question précédente pour un additionneur à retenue bondissante.



Principe :

si $X_i = Y_i = 0$ alors $c_{i+1} = 0$

si $X_i = Y_i = 1$ alors $c_{i+1} = 1$

si $X_i = Y_i$ alors $c_{i+1} = X_i \rightarrow$ on n'a pas besoin d'attendre de la calculer

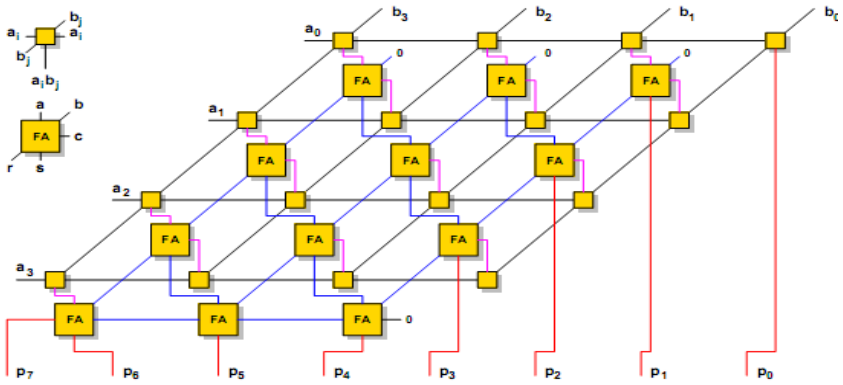
si $\forall k \in \{i, \cdots, i+p\}$, $X_i \neq Y_i$ alors $c_{i+p+1} = c_i$

si on localise une chaine de bits de telle sorte $\forall k \in \{i, \cdots, i+p\}$, $X_i \neq Y_i$, alors la retenue sortante du bloc est égale à sa retenue entrante

Multiplieur

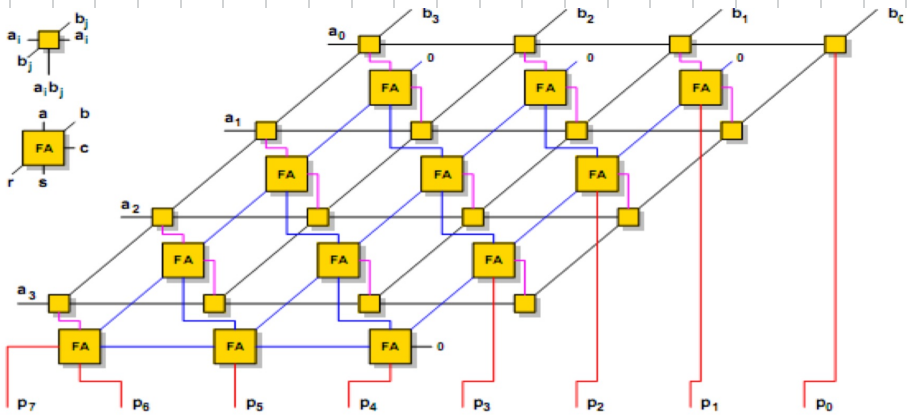
4.3 – Rappeler plusieurs principes pour réaliser une multiplication.

Rappel : multiplieur combinatoire

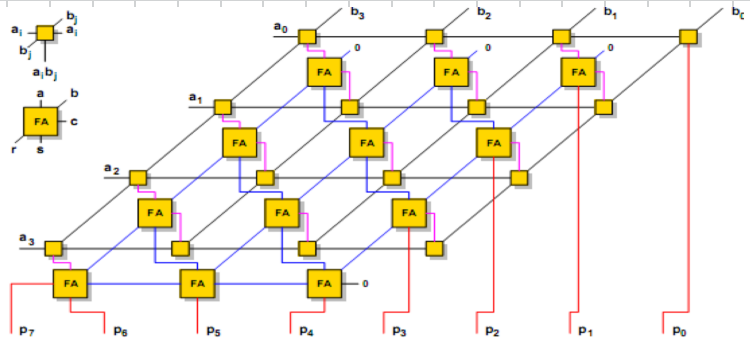


4.4 – En déduire plusieurs architectures d'un multiplieur. Exprimer les équations logiques. Evaluer les performances des architectures proposées puis comparer.

Multiplieur :



Equation : Cell : FA.


a_i b_j a_i b_j $a_i b_j$
a b FA c r s
a_0 b_3 b_2 b_1 b_0
a_1
a_2
a_3
FA
0
p_7 p_6 p_5 p_4 p_3 p_2 p_1 p_0

Max 5000.

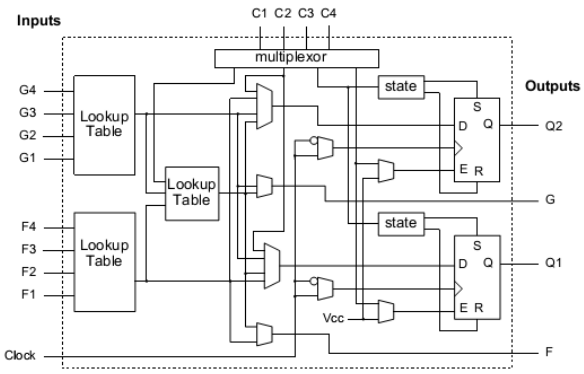
Université de Nice-Sophia Antipolis
Master 1 ESTel
Composants logiques programmables

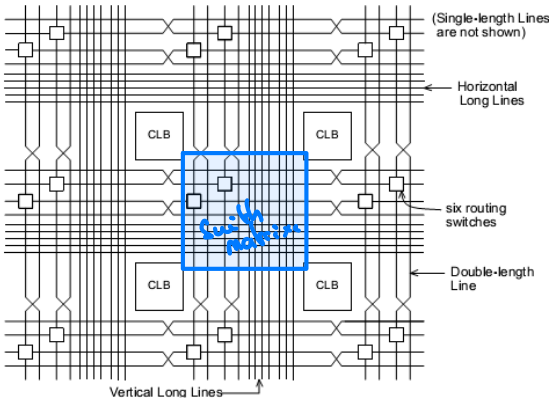
TD2 : Composants logiques programmables CPLD ou FPGA

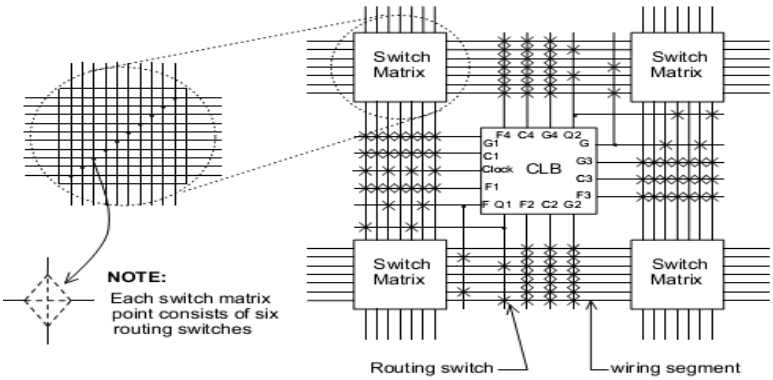
Exercice 1 : questions relatives au cours
1. Expliquer la différence entre une technologie EPROM, SRAM et Antifusible.
2. Pourquoi les délais sont plus prévisibles pour les CPLDs que les FPGAs ?
3. Expliquer la méthode de synthèse pour des composants logiques programmables de type CPLD ?
4. Question identique à la précédente pour des FPGA construit sur la base de cellules élémentaires avec LUT ou MPX.
5. Comparer la surface de silicium d'une cellule avec LUT et avec MPX.

Exercice 2 : Etude de FPGA

On donne ci-dessous un exemple de cellule élémentaire (XC4000) et leur architecture d'interconnexion.







Donner les caractéristiques principales de cette architecture. Expliquer leur impact sur la synthèse.
Reprendre cette question pour la famille virtex :

TD n°2 :

Exercice 1 : EPROM : fonction toute seule 1 fois.

SRAM : grande surface, mémorise pas. + lourd d'un EPROM

Flash : petit embarqué.

SRAM : Reconfigure de nombreuses.

Délais : tps cells :

tps routage :

méthode : 1) Déterminer le nombre de cells) 1 formules

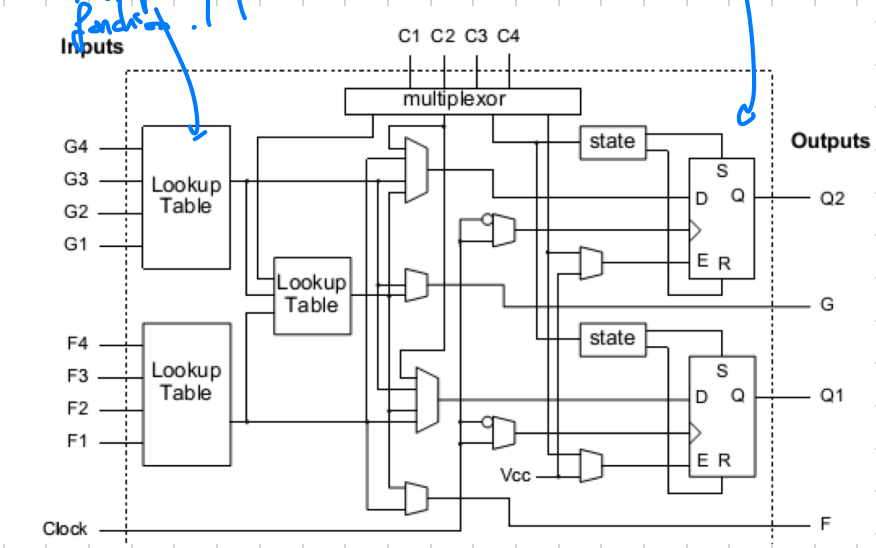
2) placement :

3) Routage :

Set, Reset, Enable : global) asynchrone.

Entrée : Composer 2 fonctions différentes.

→ Peut faire n'importe quel fonction.



Sortie en combinatoire : Bascule avec ou sans

différente étape synthèse :

- déterminer le nb de cells : (tR = ... + ?)
- placement
- routage = (TR final)

4. Pour déterminer le nb de cells sur un LUT : (Sram = 2^4 mpx 16 → 1.

synthèse de CPLDs :

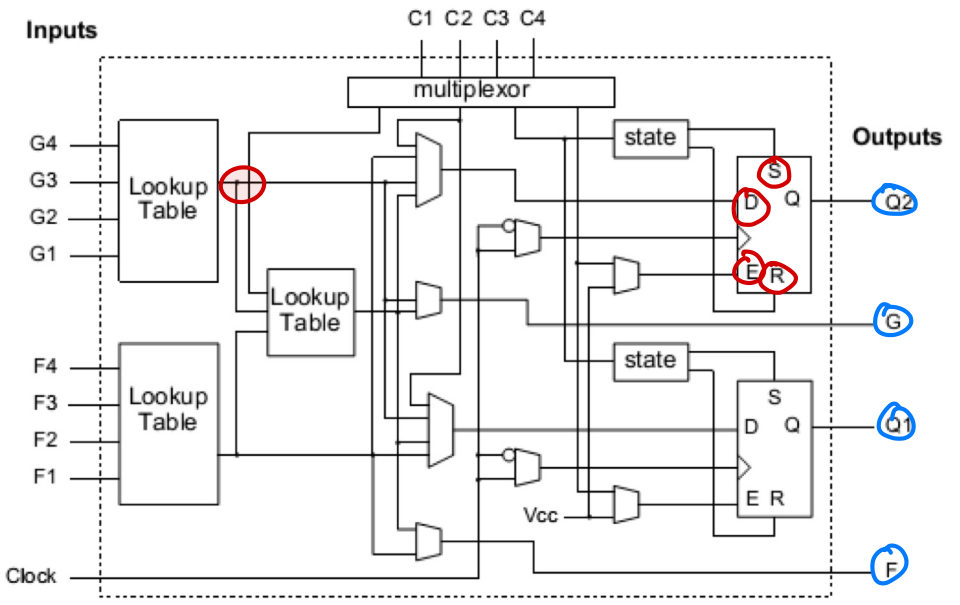
si (P < ou = à N) alors Nc = 1

si (P > N alors Nc = $\frac{(P-N)}{N-1}$ + 1

FPGA type Xilinx.

on multiplie par : par 1 entrée

FPGA type Actel : mpx → arbre binaire. (nb de cellules plus important mais plus compact)



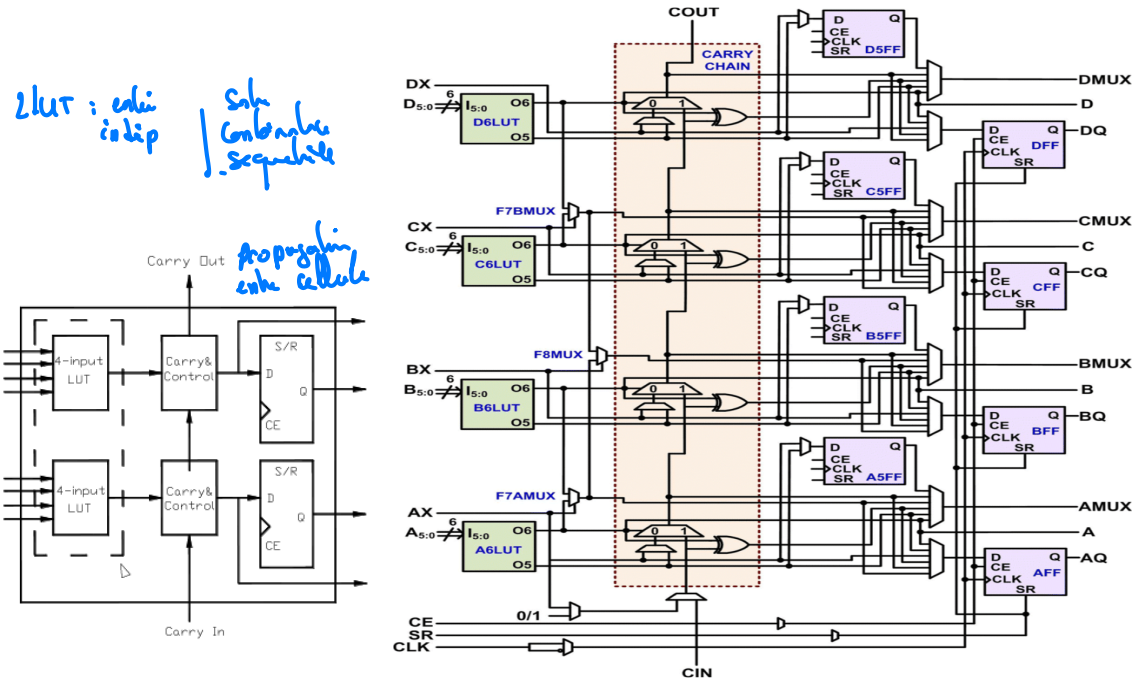
3 Lut → 3 signaux différents

2 Lut → ce c'est pour diviser le type de cellule ou pour une cellule.

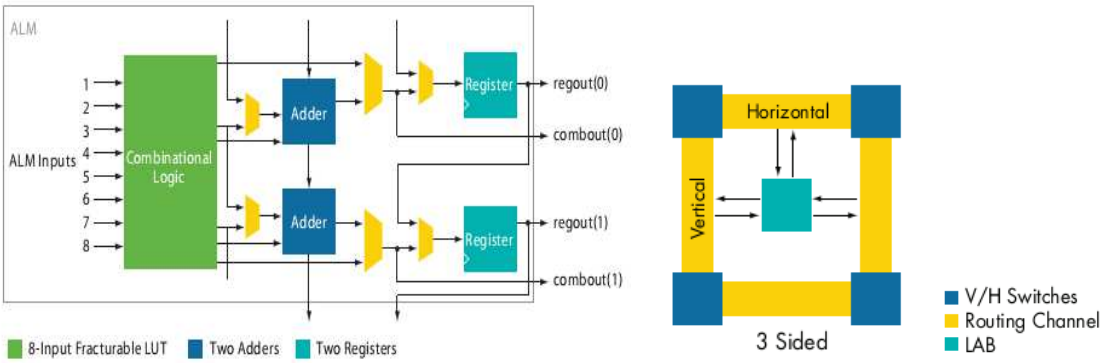
2 state → indépendants (G, F)

S, R → asynchrone :

Q_1, Q_2 → peuvent être combiner ensemble -, obligatoire sur un Registre.



CLB comme virtex 2 **Slice Virtex7**



Architecture : Altera/Intel – StratixII, cellule de type ALM (Adaptative Logic Module), plusieurs configurations : LUT6, 2x4LUT, LUT5+LUT3,…

Exercice 3 : Processeur ou FPGA ?

On souhaite réaliser la fonction suivante : Fonction $Y = A.x^2 + B.x + C$.
1 - Déterminer le temps d'exécution si cette fonction est exécutée par un processeur (on prendra un cycle par instruction).
2 - Reprendre la question précédente en utilisant un FPGA.
3- On suppose que cette fonction est calculée sur un grand nombre n d'échantillon : $Y(i) = A.x(i)^2 + B.x(i) + C$. Quels gains sont attendus sur un FPGA ?

Exercice 4 : Synthèse d'un processeur

On souhaite réaliser un core de processeur basique sur 32 bits comprenant les blocs élémentaires suivants : un compteur de programme sur 32bits (mode : compteur, clear, preset, chargement d'une valeur), quatre registres internes, une UAL avec un accumulateur un registre à décalage des multiplexeurs.
1 - Définir les fonctions relatives à chacun des composants cités ci-dessus. Etablir les équations logiques de ces composants (équations pour l'étage i).

Exercice 4 :

opération par cycle.

x^2
$A x^2$
$B x$
$A x^2 + B x$
$A x^2 + B x + C$

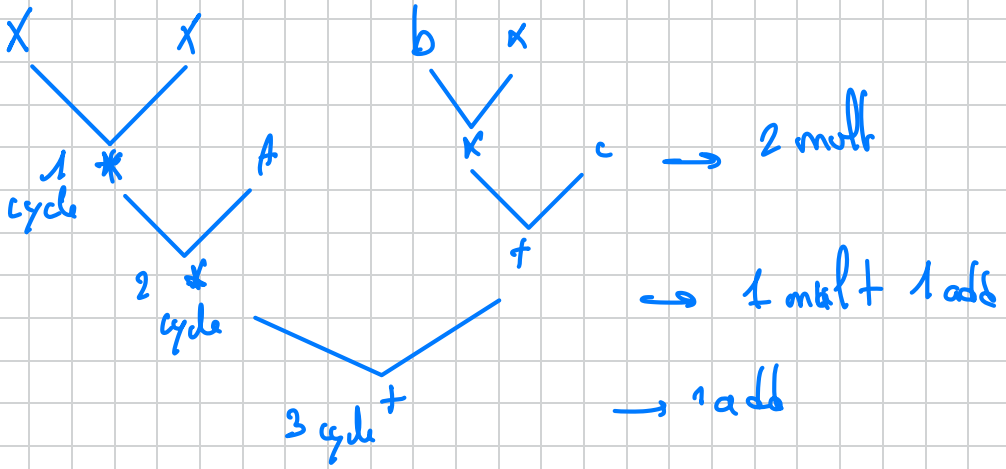
1
2
3
4
5

cycle

FPGA → cherche parallélisme.

$A x^2 + b x + c$:

archi // :



→ 2 mult

→ 1 mult + 1 add

→ 1 add

on a besoin de
2 mult + 1 add.

Altera Cpld → 8 produits
Xilinx LUT → 4 entrées
Actel mux → 4 → 1.

les 3 méthodes.

add → LUT : (toolbox minimale)) les pire avec Add

F_1	F_0	S
0	0	A and B
0	1	A or B
1	0	A + B
1	1	S = S + A

	Produit
$A \cdot B$	1
$A + B$	2
$A_i \oplus B_i \oplus R_{i-1}$	4
$A_i \oplus S_i \oplus R_{i-1}$	4

Cpld : N produits

$A \oplus B$ → 4. car $\bar{A_i} \bar{B_i} R_{i-1} + \bar{A_i} B_i \bar{R_{i-1}} + A_i \bar{B_i} \bar{R_{i-1}} + A_i B_i R_{i-1}$

max 7000 → 8 produits

2 cells car (8 + 3).



Tcell : 3 ns exemple

méthode : FPGA - Lut - 4.

6 entrées : $(S_i, F_1, F_0, A_i, B_i, R_i)$.

$$S_i^+ = \underbrace{\bar{F}_1 \bar{F}_0 A_i \cdot B_i}_{\rightarrow S_i'} + \bar{F} F_1 (A_i \oplus B_i) + F_1 F_0 (A_i \oplus B_i \oplus R_{i-1}) + F_1 F_0 (A_i \oplus S_i \oplus R_{i-1})$$

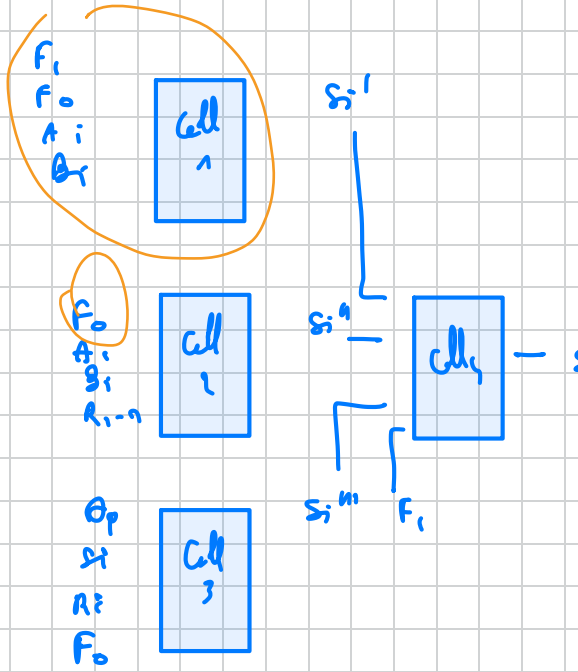
S_i'' S_i'''

$$C_i^+ = S' + F_1 \left(\bar{F}_0 (A_i \oplus B_i \oplus R_{i-1}) + F_0 (A_i \oplus B_i \oplus R_{i-1}) \right)$$

pas possible.

$S_i^+ = S_i' + F_1 (S_i'' + S_i''')$

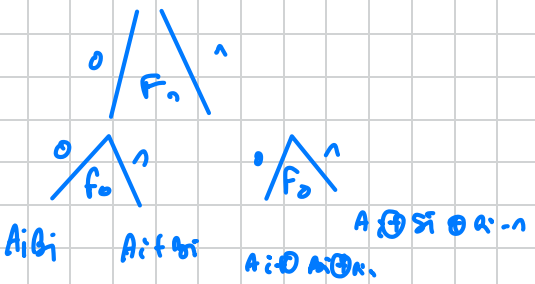
on obtient 4 entrées.



$t_R = 2 \times t_{cell} + ?$.

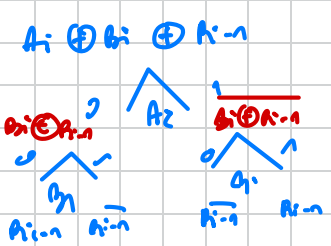
$t_{Lut} = 2$ ms exemple

FPGA : mux 4 → 1



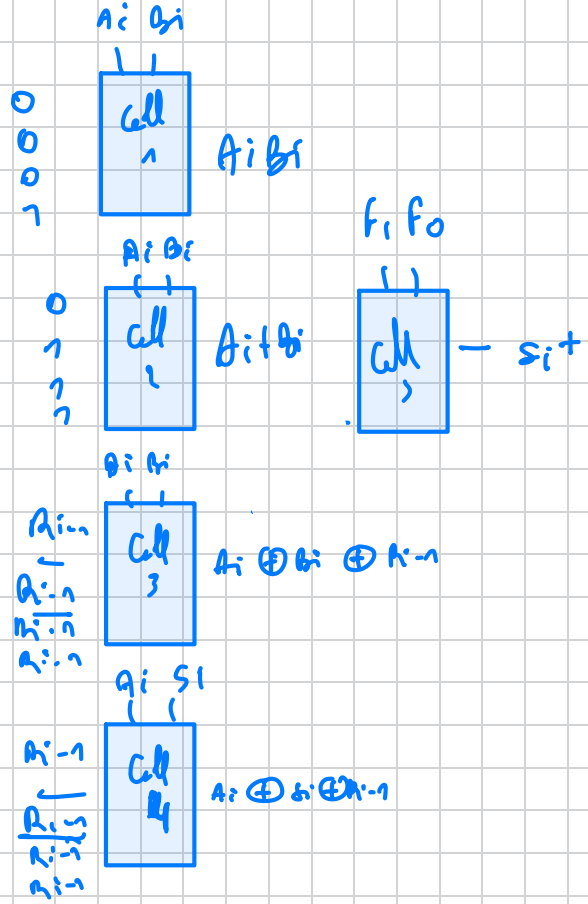
$$S_i^+ = \bar{F}_1 \bar{F}_0 A_i \cdot B_i + \bar{F} F_1 (A_i \oplus B_i) + F_1 F_0 (A_i \oplus B_i \oplus R_{i-1}) + F_1 F_0 (A_i \oplus S_i \oplus R_{i-1})$$





$A_i \oplus B_i \oplus R_{i-1}$

A_{i-1} $\bar{A}_{i-1}$ $\bar{R}_{i-1}$ R_{i-1}



$t_q = 2 \times t_{ca} + ?\ldots$

$t_q = 0{,}8$ ns exemple

Altera Cpld → 8 produits

Xilinx LUT → 4 entrées

Actel mux → 4 → 1.

reste 3 cellules.

La même pour un Compteur :

$Q_i^+ = Clear\ (\bar{Q}_i\ P_{i-1} + Q_i\ \bar{P}_{i-1} + Preset)$

$P_i = Q_i\ P_{i-1}$

Cpld : N produits

max 7000 → 8 produits

1 cells (4).

Compteur 32 bits → 64 cells.

$\bar{Q}_i\ P_{i-1}$	1
$Q_i\ \bar{P}_{i-1}$	1
Preset	1
Clear	1

Q_i^+ = 1 cell (3 produits

P_i = 1 cell.

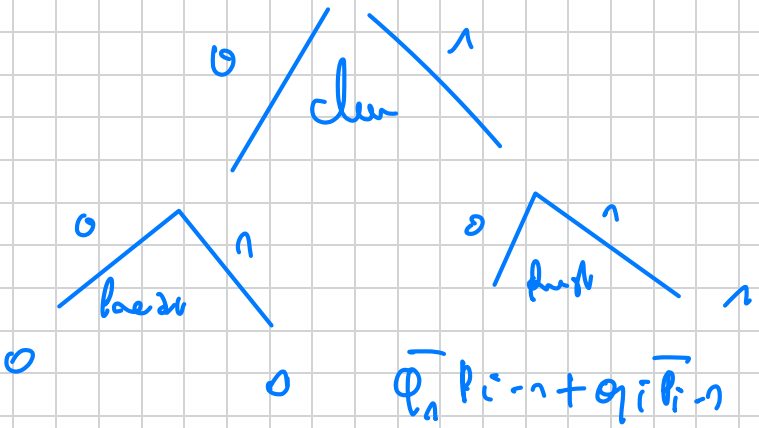
méthode : FPGA - Lut - 4.

Q_i^+ = 1 cells car (Q_i, P_i, Reset, clear).

P_i = 1 cells

FPGA : mux 4 → 1

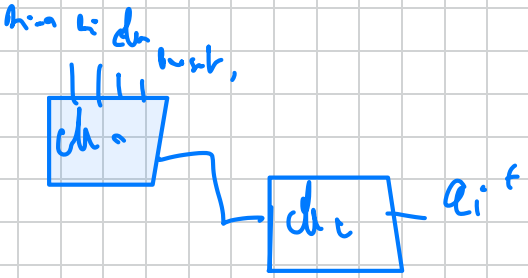
$$Q_i^+ = \text{clear} \left(\overline{Q_1} P_{i-1} + Q_i \overline{P_{i-1}} + \text{Reset} \right)$$



$\overline{Q_1} P_{i-1} + Q_i \overline{P_{i-1}}$

$\overline{Q_i} P_{i-1} + Q_i \overline{P_{i-1}}$





$TR = 2 t_{cell} + t_{c...}$

Registre à décalage :

$$Q_i^+ = (\overline{F_1} \overline{F_0} Q_i + \overline{F_1} F_0 E_i + F_1 \overline{F_0} Q_{i+1} + F_1 F_0 Q_{i-1} + \text{Reset}) \text{ clear}$$

Q_i^{+1} Q_i^{+1}

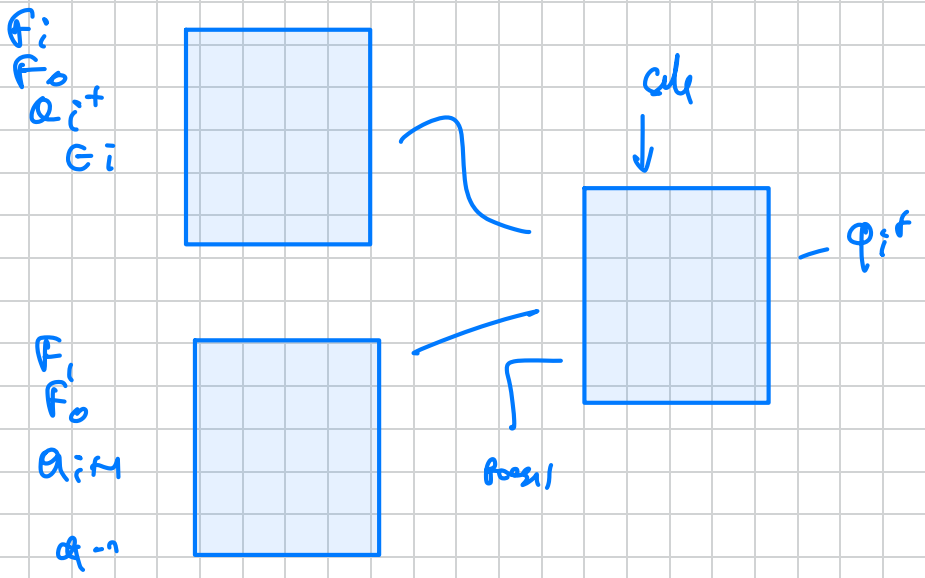
on a 5 produits :

$Q_i^+ = (Q_i^{+1} + Q_i^{+1} + \text{Reset}) \text{ clear}$

Cpld → Max 7000 - 8 produits :

1 cell.

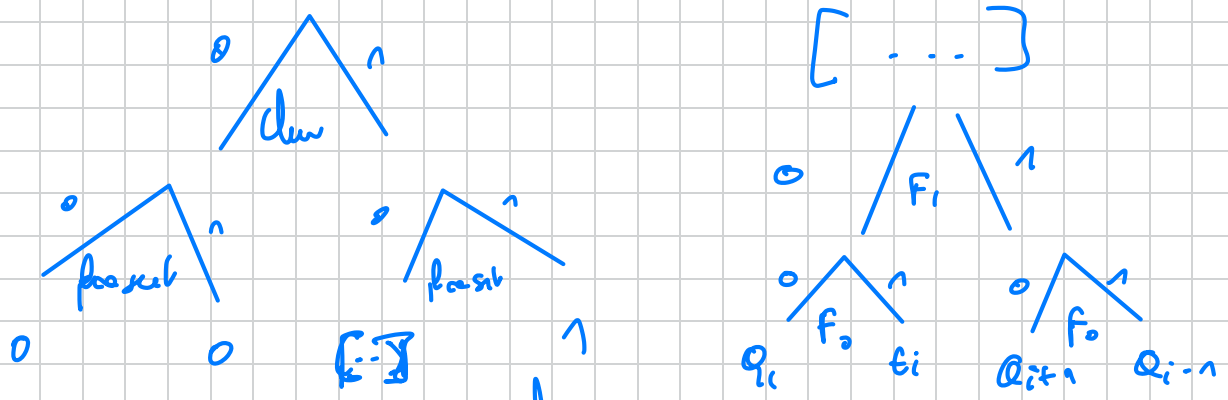
FPGA - Lut 4 → 8 entrées.

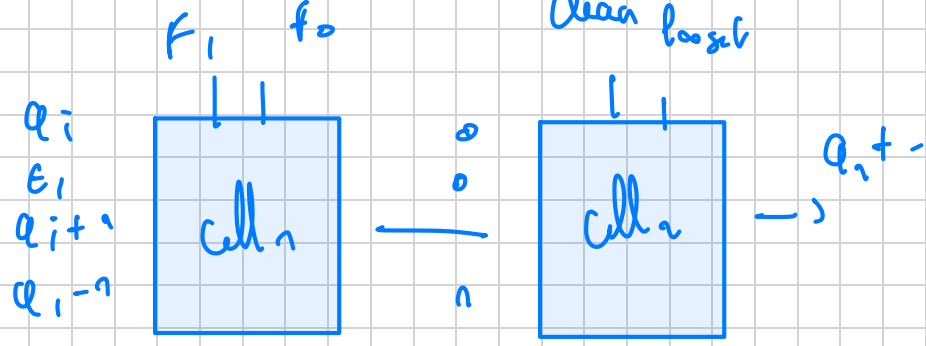


multiplexeur i

$$Q_i^{t+} = \overline{F_1}\,\overline{F_0}\,Q_i^t + \overline{F_1}F_0\,E_i + F_1\overline{F_0}\,Q_{i+1} + F_1F_0\,Q_{i-1}) + \text{Reset}) \cdot \text{clear}$$

Reset



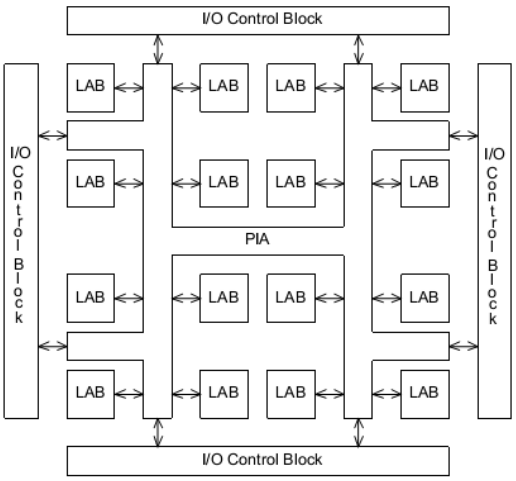


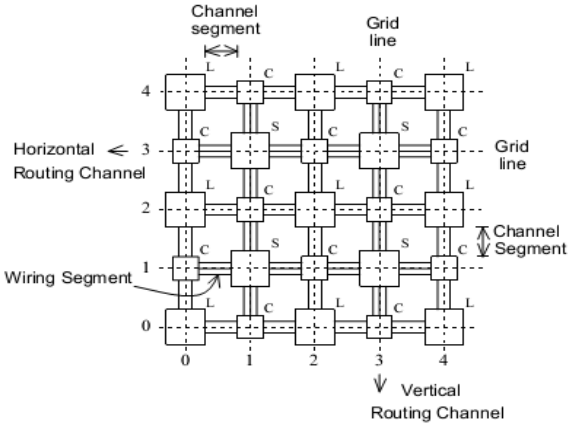
2 - Evaluer le nombre de cellules (CE) nécessaires pour un processeur 32 bits lorsque les composants suivants sont utilisés :
• CE avec LUT à 4 entrées (exemple circuit xilinx),
• CE avec MPX4->1 (exemple circuit ACTEL),
• CPLD de type Max7000 (circuit ALTERA).
Estimer le temps de réponse de ces solutions sans tenir compte des communications entre les cellules.

Exercice 5 :
Pour certains composants du processeur précédent, additionneur, compteur et shift register sur 4 bits :
1- Déterminer le nombre de cellules élémentaires pour un CPLD puis pour un FPGA
2- Choisir le composant adéquat, en supposant des matrices de cellules n*n
3 - Effectuer le placement des cellules puis le routage. Evaluer le temps de réponse des systèmes synthétisés.

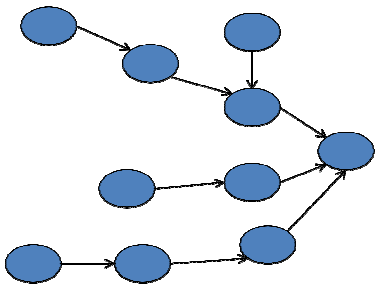
Caractéristiques des circuits :
CPLD : T_{ce} = 3ns, T_{PIA}=0,5ns, cluster composés de 16 CEs
FPGA 4-LUT : T_{ce} = 2 ns, T_{SM}=0,3 ns
FPGA - mpx : Tce= 0,8ns, Tline=0,1ns

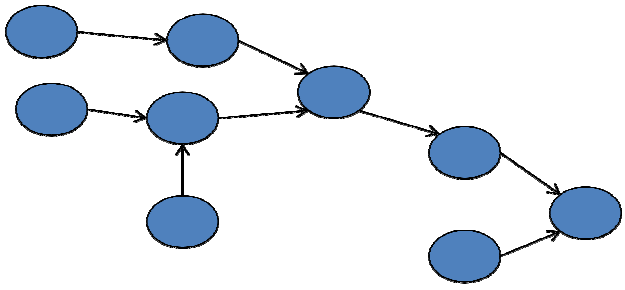
Pour rappel :



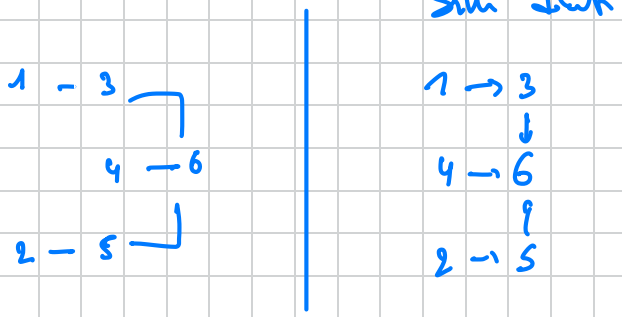


PIA : Programmable Interconnect Array
LAB : Logic Array Block
4 - Reprendre la dernière question en utilisant un placement de type « Maze routing » pour les designs suivants :





F_1 = Sous :



→ Synthèse CPLD / FPGA.

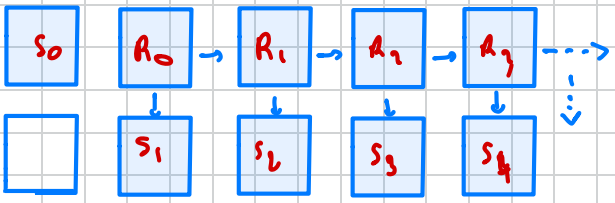
1) Déterminer le nb de cells
2) Placement.
3) Routage

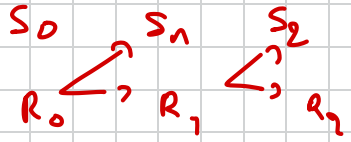
add 8 bits : Placement :

formule : $S_i = A_i \oplus B_i \oplus R_{i-1}$ ⟶ 3 entrées → 1 cell

$R_i = A_i B_i + A_i R_{i-1} + B_i R_{i-1}$ ⟶ 3 entrées → 1 cell

FPGA - 4 - LUT.



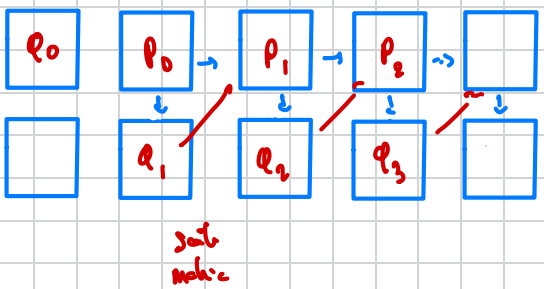


$t_R = n\, t_{R\,cell}$.

Compteur : $Q_i^+ = P_{i-1}\,\overline{Q_i} + \overline{P_{i-1}} \cdot Q_i$

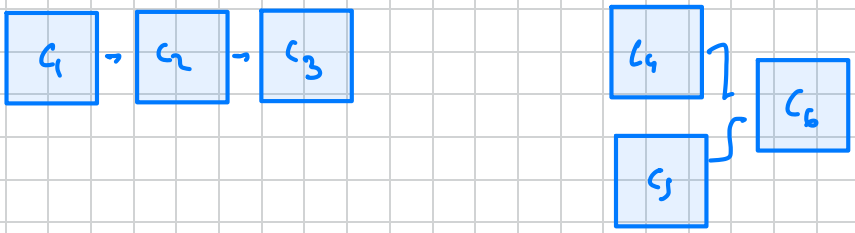
$P_i = P_{i-1} \cdot Q_i$





$t_R = n\, t_{cell} + t_{com_0}$.

F_1, F_2 F_2.



$t_R = 3 t_{cells}$ C1 → C2 → C3

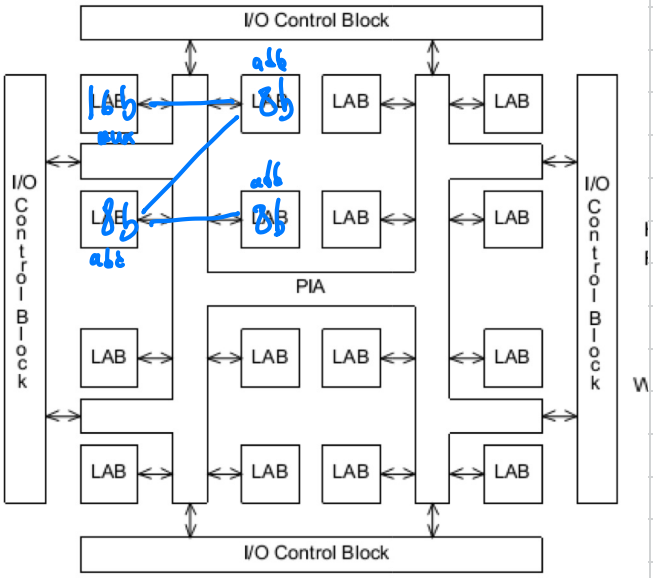
$t_R = 2 t_{cells}$ C4 → C6 ← C5

Placement sur une table de cellules.
2 fonctions différentes.

EXERCICE 5 :

CPLD → sont dans les LAB.

Pour rappel :



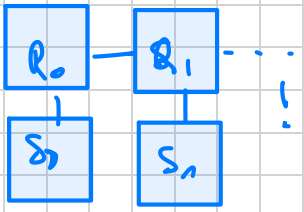
adr 32 bits

Si ; R :

$t_R = 32 \times t_{cell} + 3 \times t_{con}$

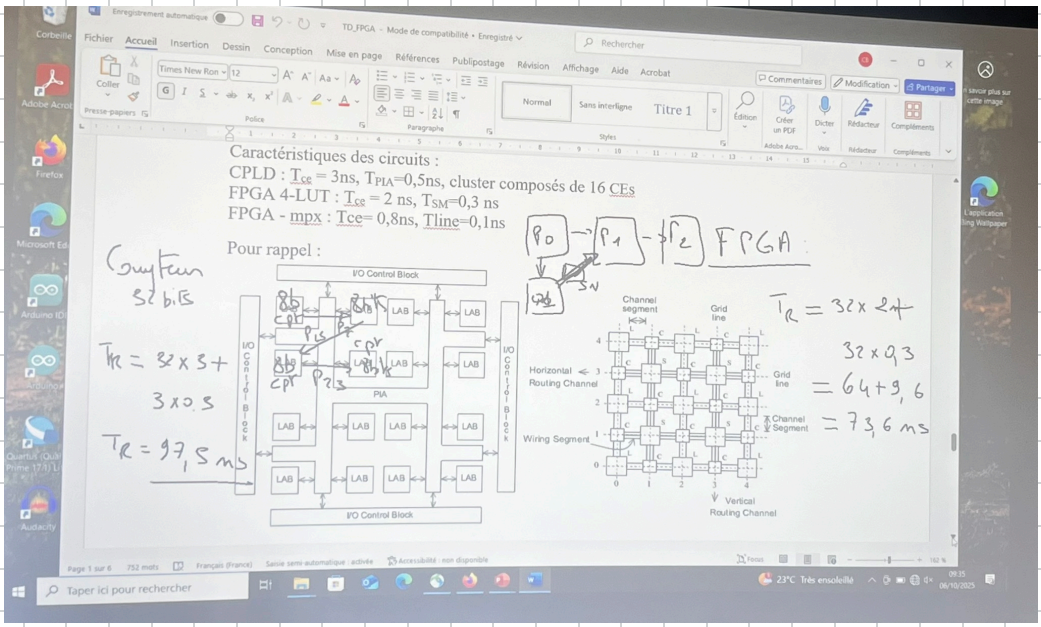
$t_R = 96 + 1,5 = 97,5$ ns.

FPGA :



$t_R = 32 \times t_{cell}$

$t_R = 64$ ns.

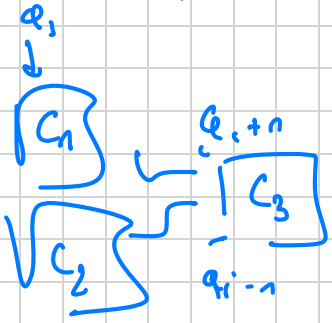


Shift. Register :

32 bits : $Q_i^+ = (\bar{F_1}\bar{F_0} Q_i + \bar{F_1} F_0 \; \varepsilon_2^0) + (F_1 \bar{F_0} Q_{i-1} + F_1 F_0 Q_{i+1})$

C_1 C_2

4-LUT :



1 bit.

LAB => 16 CE

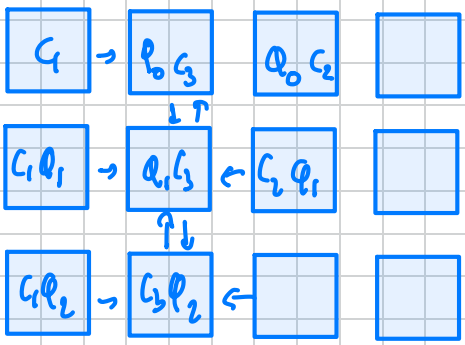
4 ≤ bits

32 bits → 2 LAB.

$t_R = (7 \times 2 t_{cell}) + 6 \times t_{com}$

$t_R : 14 \times 3 + 6 \times 0,5 = 45$ ns.

même si chaque LAB a 8 bit tout se fait en //.



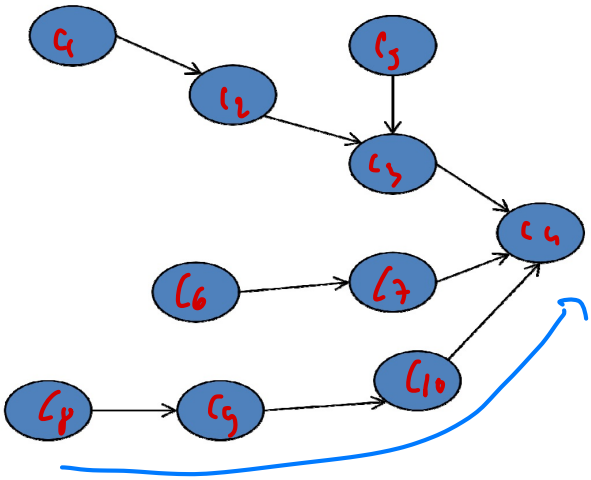
Q_1

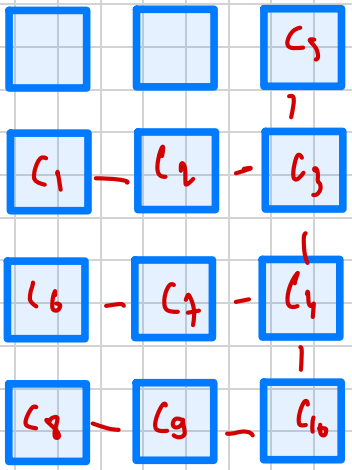
$C_3(Q_0, Q_0)$

32 bits : $t_R = 32 \times t_{cell}$.

Pour add / sous ↑↓

Pour les mux ← →.

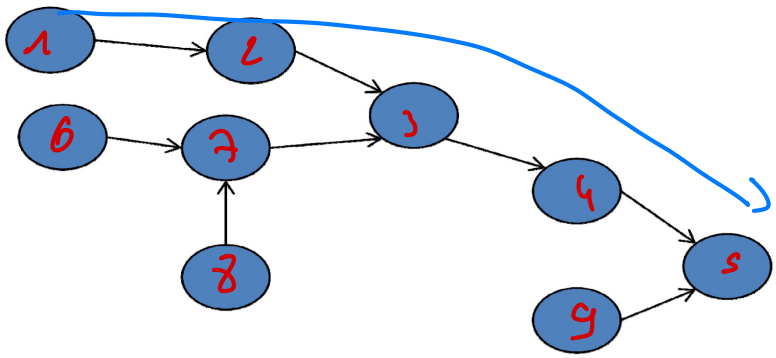


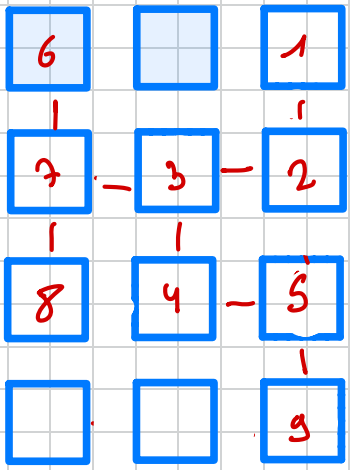


fh = 4 x fcall

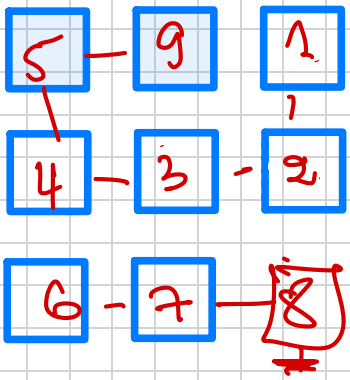
4 dll = chemin le plus

Structure diamant.





fh = 5 fcall



4.1 – Lister les process et indiquer les fonctionnalités réalisées

Le système utilise exactement les mêmes mécanismes que dans ton TP2 (affichage VGA + traitement vidéo).
D'après ton rapport, voici les processus **attendus** :

---

1) Processus de division d'horloge

(cf. TP2 — division de 50 MHz → 25 MHz)

- Génère le pixel clock (25 MHz) indispensable au VGA.
- Synchronise tout le pipeline vidéo.

---

2) Processus de balayage horizontal et vertical

(cf. TP2 : pixel_x, pixel_y, end_of_line, end_of_frame)

- Génère les compteurs H/V (0..799 et 0..524).
- Gère la position du pixel dans la trame.

---

3) Processus de synchronisation VGA

(cf. TP2 : hsync, vsync définis par les intervalles)

- Produit hs et vs selon les timings VGA.
- Assure un affichage stable et conforme 640×480@60Hz.

---

4) Processus "zone visible"

(cf. TP2 : enable = '1' si pixel_x<640 et pixel_y<480)

- Active le traitement uniquement dans la zone d'affichage.
- Désactive en dehors (fond noir).

---

5) Processus d'accès mémoire / lecture des pixels

(cf. TP2 : adressage ROM : adresse = pixel_y*420 + pixel_x)

- Calcule l'adresse de la ROM pour récupérer le pixel courant.
- Produit R,G,B pour le pixel affiché.

---

6) Processus de conversion en niveaux de gris

(décrit dans ton rapport comme filtre simple possible)

- Transforme l'image RGB en une image à un seul canal.
- Prépare l'entrée des filtres Sobel.

---

7) Processus du filtre Sobel X et Sobel Y

(cf. TP2 : Partie "Traitement vidéo – idée de conception")

- Nécessite 3 lignes de pixels → *line buffers* (principe donné).
- Applique la convolution 3×3 pour détecter les contours horizontaux et verticaux.

---

8) Processus de recombinaison des deux filtres

(cf. TP2 : "reconstituer l'image à partir des sorties des 2 filtres")

- Combine |Sx| et |Sy| pour produire l'image finale des contours.

4.2 – Peut-on améliorer les performances ? Comment ? Gain attendu ?

Oui.
D'après ton rapport TP2 (partie "traitement vidéo"), les améliorations proviennent de :

✔ 1) Pipeline complet du traitement vidéo

(cf. TP2 : pipeline d'affichage, signaux synchronisés)

- Les étapes (lecture, grayscale, Sobel X, Sobel Y, recombinaison) peuvent être pipelinées.
- Chaque étape utilise l'horloge pixel_clk pour sortir 1 pixel/cycle.

✔ 2) Parallélisation naturelle sur FPGA

Ton rapport mentionne les possibilités d'ajouter des filtres sans changer l'architecture.
Grâce au FPGA :

- Les 9 opérations du masque Sobel peuvent être faites en **parallèle**.
- Le CPU ferait ces opérations **séquentiellement**.

✔ 3) Utilisation de ressources internes

(cf. TP2 : bloc ROM, pipeline RGB → extension possible)

- Le FPGA dispose de **multiplicateurs/DSP** capables d'exécuter les 9 multiplications simultanément.
- Les additions se font en cascade dans les LUT/FF du pipeline.

✔ Gain attendu

Un filtre Sobel = 9 multiplications + additions → séquentiel sur CPU → lent.
Sur FPGA → **1 pixel par cycle**.

Gain typique :
➡️ **×10 à ×20** selon la fréquence et le CPU de référence.

Version courte à mettre dans la copie (basée 100 % sur ton TP)

4.1
Les processus sont :
division d'horloge, compteurs H/V, synchronisations VGA, zone visible, adressage ROM, conversion en niveaux de gris, filtres Sobel X et Y (avec line buffers), recombinaison finale.
Ces processus assurent l'acquisition du pixel, la formation de la fenêtre 3×3 et la génération de l'image de contours.

4.2
Oui. Les performances sont améliorées grâce à une architecture parallèle et pipelinée : les multiplications du masque Sobel sont exécutées simultanément et chaque étape du traitement produit 1 pixel par cycle.
Cela donne un gain d'un facteur **10 à 20** par rapport à un traitement séquentiel sur